

队列专题训练

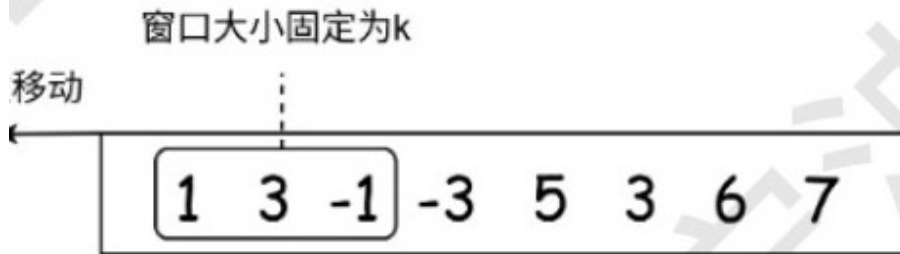
1. 某队列初始状态为升序序列（序列中的元素互不相同），经过若干次“将队首元素移至队尾”的操作后，所有元素依次出队得到序列 a。编写 Python 程序，在序列 a 中查找目标值 key 的位置，部分代码如下：

```
i, j = 0, len(a) - 1
ans = -1
while i <= j:
    m = (i + j) // 2
    if a[m] == key:
        ans = m
        break
    if _____:
        if a[i] <= key < a[m]:
            j = m - 1
        else:
            i = m + 1
    else:
        if a[m] < key <= a[j]:
            i = m + 1
        else:
            j = m - 1
```

则程序中划线处的代码应为（ ）

- A. $a[i] \leq a[m]$ B. $a[m] \leq a[j]$ C. $a[i] < a[m]$ D. $a[i] \leq a[j]$

2. 有一个数字传送带每秒匀速向左移动 1 个数字，如第 12 题图所示。其上方有观察数字传送带的固定窗口，宽度为 k（即可以看到 k 个数字）。操作员需每秒一次记录窗口中的最大值，例如图中的数字每秒记录下来的结果依次为“3 3 5 5 6 7”。现在已知数字传送带的所有数据 nums 和窗口宽度 k，操作员编写了记录每秒窗口最大值的 Python 代码如下，方框中应填入的正确代码为（ ）



```
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = int(input()) # 输入固定窗口大小 k
n = len(nums)
q = [0] * n
head, tail = 0, 0
ans = []

# 处理前 k 个元素
for i in range(k):
    while head < tail and nums[i] >= nums[q[tail - 1]]:
        tail -= 1
    q[tail] = i
    tail += 1
ans.append(nums[q[head]])
```

```

# 处理剩余元素
for i in range(k, n):
    while head < tail and nums[i] >= nums[q[tail - 1]]:
        tail -= 1
    q[tail] = i
    tail += 1
    # 移除超出窗口范围的队列头部元素
    # 方框中填入代码
print(ans) # 若 k 为 3, 输出应为 [3,3,5,5,6,7]

```

A.	while tail < head and q[head] <= i - k: head += 1 ans.append(nums[q[tail]])	B.	while tail < head and q[tail] <= i + k: tail += 1 ans.append(nums[q[head]])
C.	while head < tail and q[tail] <= i - k: tail += 1 ans.append(nums[q[tail]])	D.	while head < tail and q[head] <= i - k: head += 1 ans.append(nums[q[head]])

3. 队列初始为空, 字符 A、B、C、D、E、F 按某一初始顺序 P 依次入队, 入队规则为: 若队列为空, 字符入队; 否则字符入队后再将队首元素出队并再入队。所有字符入队完成后, 队列从队首到队尾的字符顺序为 CEAFDB, 则在初始顺序 P 中, 从 1 开始计数, 排在第 4 个的字符为()

A. C B. E C. F D. A

4. 某队列中, 队首到队尾的元素依次为 A、B、C、D、E、F, 元素经过一系列的出队或出队再入队后, 队列中的元素依次为 E、A、C, 则元素 A 至少经历了多少次出队再入队? ()

A. 1 B. 2 C. 3 D. 4

5. 有如下 Python 程序段:

```

s = "ABCDEF"; head = 0; tail = 0
que = [""] * 10
for i in range(len(s)):
    if i % 2 == 0:
        que[tail] = s[len(s)-1-i]
    else:
        que[tail] = s[i-1]
    tail = tail + 1
for i in range(len(s)):
    print(que[head], end="")
    head = head+1

```

该程序运行后输出的结果为 ()

A. FAEBDC B. FADCBE C. FBDDBF D. FDBACE

6. 有如下 Python 程序段:

```

a = [1,1,0,1,0,0,1,1,1,0,1]
k = 2
ans = 0
num = 0
i = 0
for j in range(len(a)):
    if a[j] == 0:
        num += 1
    while num > k:
        if a[i] == 0:
            num -= 1
        i += 1
    if j - i + 1 > ans:
        ans = j - i + 1
print(ans)

```

执行程序后, 输出的结果为 ()

- A. 3 B. 4 C. 5 D. 6

7. 某翻译程序使用容量为 m 的内存缓存单词译义，若缓存已满，则删除最早进入内存的单词。内存中每个单词节点为 $[key, val, next]$ ，同首字母单词用链表连接， $qhead$ 、 $qtail$ 按进入内存的先后顺序维护 FIFO 队列。如下程序用于在外存查找成功后插入新单词，划线处代码依次应为（ ）

```
def search_memory(key):
    p = _____①_____
    while p != -1:
        if memory[p][0] == key:
            return memory[p][1]
        p = memory[p][2]
    return 'fail'

def ins_memory(key, val, m):
    global qhead, qtail
    if qtail - qhead >= m:
        for i in range(26):
            if head[i] == qhead:
                _____②_____
                break
            qhead += 1
    memory[qtail] = [key, val, -1]
    num = ord(key[0]) - 97
    if head[num] == -1:
        head[num] = qtail
    else:
        p = head[num]
        while memory[p][2] != -1:
            p = memory[p][2]
        memory[p][2] = qtail
    qtail += 1
```

- A. ① $head[ord(key[0]) - 97]$ ② $head[i] = memory[qhead][2]$
 B. ① $qhead$ ② $head[i] = qhead + 1$
 C. ① $head[ord(key[-1]) - 97]$ ② $memory[qhead][2] = head[i]$
 D. ① $qtail$ ② $head[i] = -1$

8. 某监测系统需找出异常波动点最密集连续 1 小时时段。一天的数据按时间顺序存储在列表 a 中，每 5 分钟采集一次，因此连续 1 小时对应 12 个采集时刻。 $flag[i]$ 表示第 i 个时刻是否异常。如下程序用于输出异常点最多时段的起始时间，划线处代码依次应为（ ）

```
n = len(a)
flag = [0] * n
for i in range(n):
    if a[i][2] > std:
        flag[i] = 1
start_time = cnt = 0
for i in range(12):
    _____①_____
    max_cnt = cnt
    for i in range(1, n - 11):
        cnt = cnt - flag[i - 1] + _____②_____
        if cnt > max_cnt:
            max_cnt = cnt
            _____③_____
```

- A. ① $cnt += flag[i]$ ② $flag[i+11]$ ③ $start_time = i$
 B. ① $cnt = flag[i]$ ② $flag[i+12]$ ③ $start_time += 1$
 C. ① $cnt += a[i][2]$ ② $flag[i-1]$ ③ $max_cnt = i$
 D. ① $cnt += flag[i]$ ② $flag[i+12]$ ③ $start_time = i-1$

9. 数组 a 存储一系列货物重量, 卡车载重为 weight。要求选择一段连续货物, 使总重不超过 weight 且尽可能大; 若有多个总重相同的方案, 选后出现的方案。如下程序中划线处代码依次应为 ()

```
a = [20,10,50,70,25,35,30,5,80]
weight = 100
i = j = s = 0
besti = bestj = bests = 0
while ①:
    s += a[j]
    j += 1
    while i <= j and s > weight:
        ②
        i += 1
    if ③:
        bests = s
        besti = i
        bestj = j
print(bests, besti, bestj)
```

- A. ① j < len(a) ② s -= a[i] ③ s >= bests
B. ① i < len(a) ② s -= a[j] ③ s > bests
C. ① j <= len(a) ② s += a[i] ③ s >= weight
D. ① j < len(a) ② i += 1 ③ s > weight

10. 某智能终端每分钟采集一次用电功率。网络正常时, 终端先按缓存顺序上传此前暂存的数据, 再上传当前数据; 网络异常时, 终端将当前数据暂存到本地缓存队列 buffer 中, 缓存上限为 limit 条。若缓存已满, 则先删除最早暂存的数据, 再加入当前数据。如下程序中划线处代码依次应为 ()

```
buffer = []
limit = 500
while True:
    power = read_power()
    if network_ok():
        while len(buffer) > 0:
            upload(①)
        upload(power)
    else:
        if len(buffer) == limit:
            ②
            ③
# 延时 60 秒, 代码略
```

- A. ① buffer.pop(0) ② buffer.pop(0) ③ buffer.append(power)
B. ① buffer.pop() ② buffer.pop() ③ buffer.insert(0, power)
C. ① buffer[0] ② buffer.clear() ③ buffer.append(power)
D. ① buffer.pop(0) ② buffer.pop(0) ③ buffer.insert(0, power)

11. 某配送站有普通任务队列 q[0] 和加急任务队列 q[1], 两个队列都采用循环队列实现。约定 head[k] 指向第 k 个队列的队首元素, tail[k] 指向下一个入队位置; 队列空的条件为 head[k] == tail[k]; 为了区分队空和队满, 每个循环队列浪费一个存储单元。配送规则如下: 任务到达时按类型进入对应队列, 类型 0 为普通任务, 类型 1 为加急任务; 每接收 2 个任务后配送 1 个任务, 若加急队列非空则优先配送加急任务, 否则配送普通任务; 所有任务接收完毕后, 继续按该优先规则配送, 直到两个队列均为空。

(1) 若 n = 4, 任务序列 tasks = [[1,0], [2,0], [3,1], [4,1], [5,1], [6,1], [7,0], [8,0]], 其中每项依次为任务编号和任务类型, 则最终配送任务编号顺序为 _____。

(2) 实现循环队列基本操作的部分代码如下, 请在划线处填入合适代码。

```
n = 4
q = [[0] * n for i in range(2)]
head = [0, 0]
tail = [0, 0]
def empty(k):
    return head[k] == tail[k]
def full(k):
    return _____①_____
def enqueue(k, x):
    if full(k):
        return False
    q[k][tail[k]] = x
    _____②_____
    return True
def dequeue(k):
    x = q[k][head[k]]
    _____③_____
    return x
```

(3) 根据题意完成如下程序, 请在划线处填入合适代码。

```
tasks = [[1,0], [2,0], [3,1], [4,1], [5,1], [6,1], [7,0], [8,0]]
ans = []
for i in range(len(tasks)):
    x, k = tasks[i][0], tasks[i][1]
    ok = enqueue(k, x)
    if ok and i % 2 == 1:
        if _____④_____:
            ans.append(dequeue(1))
        elif not empty(0):
            ans.append(dequeue(0))
while not empty(0) or not empty(1):
    if not empty(1):
        ans.append(dequeue(1))
    else:
        ans.append(dequeue(0))
print(ans)
```

12. 某码头负责装载车辆, 每辆车有唯一的正整数编号。现因天气原因该码头仅开放一个装载点, 供车辆依次上船。装载规则如下:

I. 每过一个单位时间, 有一辆车到达并加入排队队伍, 同时有一辆车正在装载。

II. 为保证后续分类方便, 同一品牌的车辆必须在队伍中连续排列。若队伍中同时有品牌 0 和 品牌 1 的车辆, 且当前正在装载的是品牌 1 的车, 则接下来会优先让所有品牌 1 的车完成装载, 再处理品牌 0 的车。

III. 当一辆新车到达时, 如果队伍中已有同品牌的车, 则该车会直接排到该品牌所有车辆的末尾; 否则, 新车直接排到所有车辆的末尾。

现在需要计算: 当一辆车到达并完成排队后, 它的前方还有多少车辆在等待。

(1) 假设在码头开放前, 已经有车辆 $[[1,0], [2,1], [3,2], [4,1], [5,0], [6,2]]$ 按序到达, 列表中的每个元素包含两个数据项, 依次为车辆编号和品牌。码头开放后, 车辆 $[[7,0], [8,0], [9,2], [10,2], [11,1]]$ 依次到达。例如, 当车辆 $[7,0]$ 到达时, 7 号车可以直接排到 0 号品牌队列的队尾, 此时 7 号车辆的前方还有 2 辆车 (正在装载的 1 号和 5 号车)。则当车辆 $[11,1]$ 到达时, 11 号车辆的前方还有 _____ 辆车。

(2) 定义如下 arrive(car, brand) 函数，其功能是处理一辆新车到达码头时的排队并计算该车前方的车辆数量。参数 car 表示车辆编号，brand 表示品牌。data[brand] 表示该品牌车辆队列，n 表示每个品牌队列数组容量；该队列采用普通顺序队列，指针不循环复用；head[brand]、tail[brand]、length[brand] 分别表示该品牌队列的队首下标、下一个入队位置和当前长度；length[brand] == 0 表示该品牌队列为空，tail[brand] == n 表示该品牌队列已满；order[h:t] 保存当前仍在等待的品牌顺序。请在划线处填入合适的代码。

```
def arrive(car, brand):
    global t
    if tail[brand] == n:
        return -1
    front = 0
    for b in order[h:t]:
        if b == brand:
            break
        front += length[b]
    front += length[brand]
    if length[brand] == 0:
        ①
        t += 1
    data[brand][tail[brand]] = car
    ②
    length[brand] += 1
    return front
```

(3) 提前到达的车辆和码头开放后到达的车辆信息仍存储在列表 cars 和 new_cars 中。如下 Python 程序用于模拟车辆到达排队过程，并计算每辆车到达时其前方等待的车辆数。请在划线处填入合适的代码。

```
m = 3
n = 100
data = [[0] * n for i in range(m)]
head = [0] * m; tail = [0] * m; length = [0] * m
order = [0] * n
h = t = 0

def leave():
    global h
    brand = order[h]
    car = data[brand][head[brand]]
    ①
    length[brand] -= 1
    if length[brand] == 0:
        ②
    return car

def show_queue():
    # 显示当前队伍情况，代码略
cars = [[1,0], [2,1], [3,2], [4,1], [5,0], [6,2]]
for car, brand in cars:
    front = arrive(car, brand)
new_cars = [[7,0], [8,0], [9,2], [10,2], [11,1]]
for car, brand in new_cars:
    front = arrive(car, brand)
    print('前面有' + str(front) + '辆车在队伍中')
    left = leave()
    show_queue()
```

三、答案与解析

1. 202604-绍兴-高三-期中.pdf

参考答案: A

解析: $a[i] \leq a[m]$ 用来判断左半段是否有序。若左半段有序, 再根据 key 是否落在 $[a[i], a[m]]$ 中调整边界; 否则右半段有序, 按右半段范围调整。

2. 202511-宁波一模-月考.pdf

参考答案: D

解析: 队列中保存的是可能成为窗口最大值的元素下标, 队头 $q[head]$ 是当前窗口最大值下标。处理新元素后, 要移除已经不在当前窗口 $[i-k+1, i]$ 内的队头元素, 即当 $q[head] \leq i-k$ 时令 $head += 1$ 。随后当前窗口最大值为 $nums[q[head]]$, 所以选 D。

3. 202605-县域教研-高三-月考.pdf

参考答案: A

解析: 本题依据原卷参考答案确定。复习时重点核对题干中的关键概念、限制条件和选项表述。

4. 202605-诸暨-高三-三模.pdf

参考答案: B

解析: 队首序列 A、B、C、D、E、F 经过出队或出队再入队后要留下 E、A、C。A 第一次出队再入队才能排到后面, 后续还需再经历一次出队再入队才能位于 E 后、C 前, 最少 2 次。

5. 202501-宁波九校-期末.pdf

参考答案: B

解析: 循环中依次写入队列的字符为: $i=0$ 写入 F, $i=1$ 写入 A, $i=2$ 写入 D, $i=3$ 写入 C, $i=4$ 写入 B, $i=5$ 写入 E。随后从 $head=0$ 开始顺序输出, 结果为 FADCBE。

6. 202603-杭二-期初.pdf

参考答案: D

解析: 该程序用滑动窗口维护最多包含 2 个 0 的最长连续子序列。最长窗口之一为下标 3~8: $[1, 0, 0, 1, 1, 1]$, 长度为 6, 因此输出 6。

7. 改编自 09 队列/202406-宁波九校-高二-期末-15.md

参考答案: A

解析: 内存查找按单词首字母进入对应链表, 入口为 $head[ord(key[0]) - 97]$ 。缓存满时 $qhead$ 指向最早进入内存的节点, 若它正是某首字母链表头, 需要将该链表头改成 $memory[qhead][2]$, 再整体右移 $qhead$ 。

8. 改编自 03python 基础/202604-宁波-高三-二模-14.md

参考答案: A

解析: 先统计第一个长度为 12 的窗口异常点数量; 窗口右移时减去离开的 $flag[i-1]$, 加上新进入的 $flag[i+11]$ 。只有出现更大值时才记录当前起点 i , 数量相同则保留最早时段。

9. 改编自 03python 基础/202603-金丽衢-月考-13.md

参考答案: A

解析: j 是右边界, 循环条件为 $j < len(a)$ 。当窗口和超过载重时, 从左侧移出 $a[i]$ 并右移 i 。题目要求总重相同取后出现方案, 因此用 $s \geq bests$ 更新。

10. 改编自 06 信息系统/202604-台州-高三-二模-13.md

参考答案: A

解析: 缓存队列需要保持先进先出。网络恢复后应从队首依次取出并上传, 因此用 $buffer.pop(0)$; 网络异常且缓存已满时, 也要先删除最早暂存的数据, 即 $buffer.pop(0)$; 当前采集值加入队尾, 使用 $buffer.append(power)$ 。

11. 原创综合队列题

参考答案: (1) $[1, 3, 4, 5, 6, 2, 7, 8]$; (2) ① $(tail[k] + 1) \% n == head[k]$; ② $tail[k] = (tail[k] + 1) \% n$; ③ $head[k] = (head[k] + 1) \% n$; (3) ④ $not\ empty(1)$

解析：循环队列浪费一个单元时，队满条件为尾指针再后移一位会追上队首，即 $(tail[k] + 1) \% n == head[k]$ 。入队后 $tail[k]$ 后移，出队后 $head[k]$ 后移。配送时加急队列非空则先配送加急任务，所以④为 `not empty(1)`。按题给任务序列模拟：接收 1、2 后配送 1；接收 3、4 后配送 3；接收 5、6 后配送 4；接收 7、8 后配送 5；最后继续配送 6、2、7、8。

12. 改编自 202605-镇海中学-月考.pdf

参考答案：（1）2；（2）① `order[t] = brand`；② `tail[brand] += 1`；（3）① `head[brand] += 1`；② `h += 1`

解析：本题把每个品牌的车辆直接维护为一个普通顺序队列，不再用节点指针串接车辆。`length[brand] == 0` 表示该品牌队列为空，`tail[brand] == n` 表示该品牌队列已满。新品牌第一次出现时写入品牌顺序队列 `order`；车辆入队后 `tail[brand]` 直接加 1。车辆离开时，只需让该品牌队列的 `head[brand]` 直接加 1 并减少 `length[brand]`；若该品牌队列清空，品牌顺序队列的队首 `h` 后移。按题给序列模拟，`[11, 1]` 到达时品牌 1 队列前方有 2 辆车。